

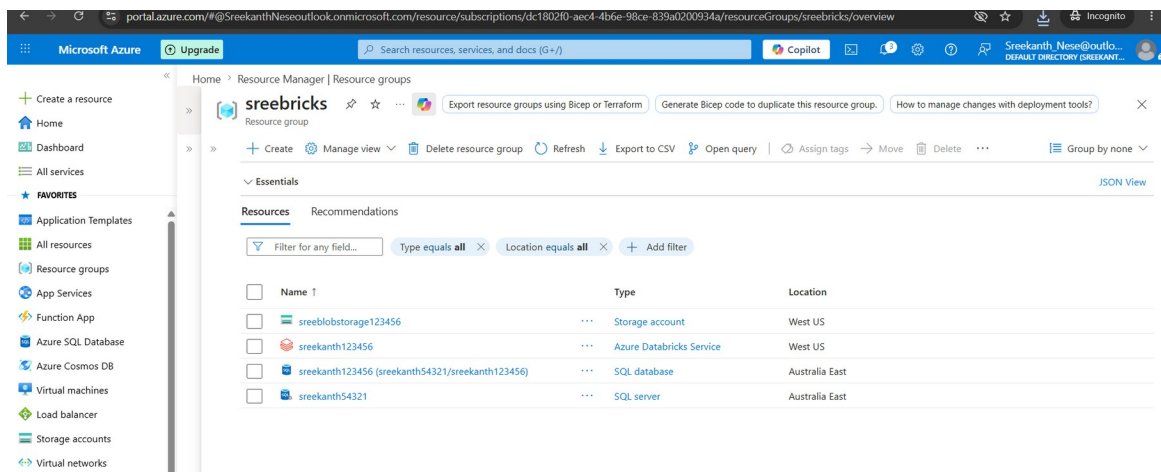
Bike Sales & Employee Training Cost Analysis

using Azure Databricks (PySpark) → Azure SQL Database

This project uses an Azure Databricks notebook (PySpark) to read raw CSV files from Azure Blob Storage, apply business transformations, and write the processed results back into Azure SQL Database as ready-to-query tables. It covers two parallel analyses run from the same notebook: bike sales profitability, and employee training cost by job title.

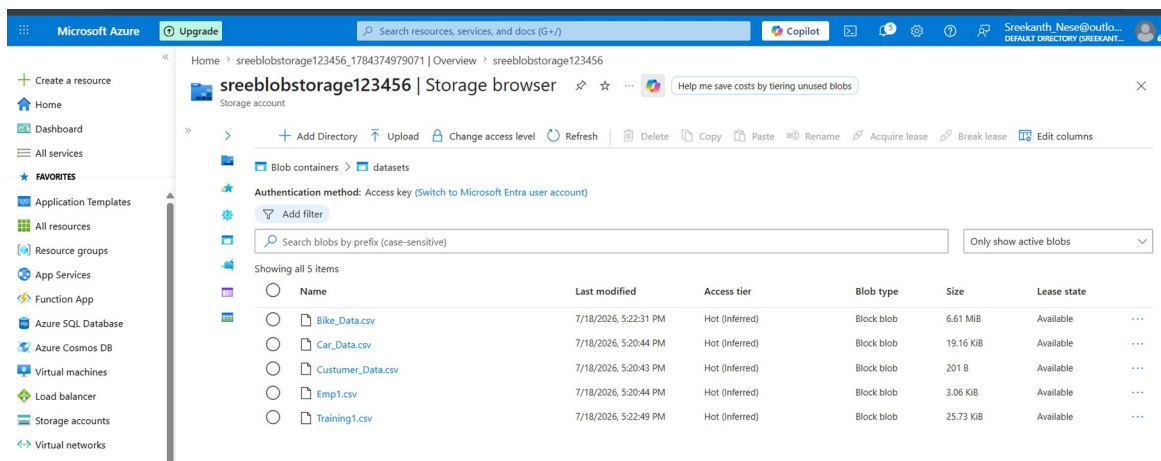
Step 1: Created the Resource Group and Core Services

A Resource Group named “sreebricks” was created to hold every service this project needs: a Blob Storage account (sreeblobstorage123456, West US) for the raw source files, an Azure Databricks Service (sreekanth123456, West US) for the PySpark processing, and an Azure SQL Server (sreekanth54321) with an Azure SQL Database (sreekanth123456), both in Australia East, as the destination for the processed tables.



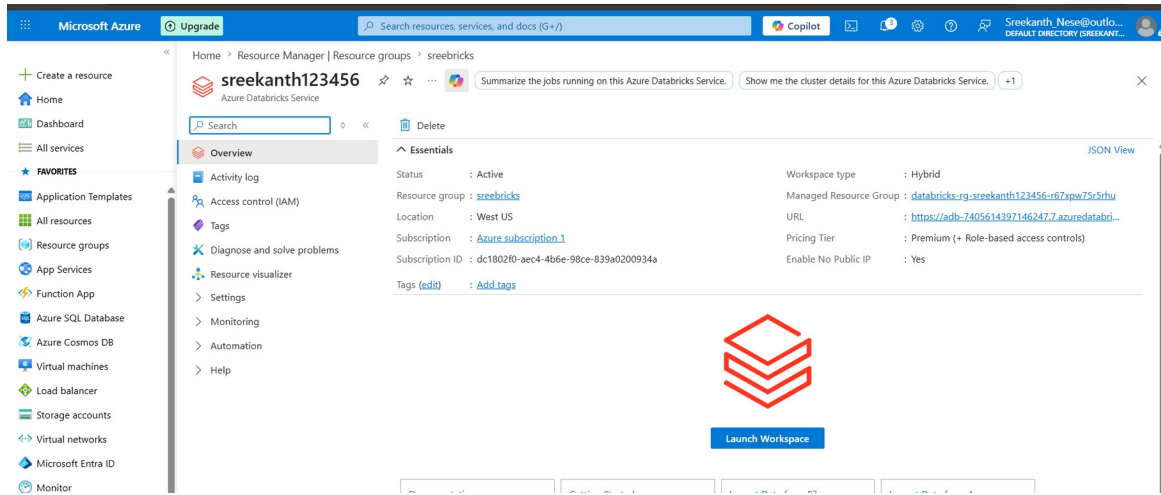
Step 2: Uploaded Source Data to Blob Storage

The datasets container in sreeblobstorage123456 holds five raw CSV files: Bike_Data.csv (6.61 MiB, the main sales fact file), Car_Data.csv (19.16 KiB), Customer_Data.csv, Emp1.csv (employee master data, 3.06 KiB), and Training1.csv (training/course cost data, 25.73 KiB). This notebook works with the Bike, Employee, and Training files.



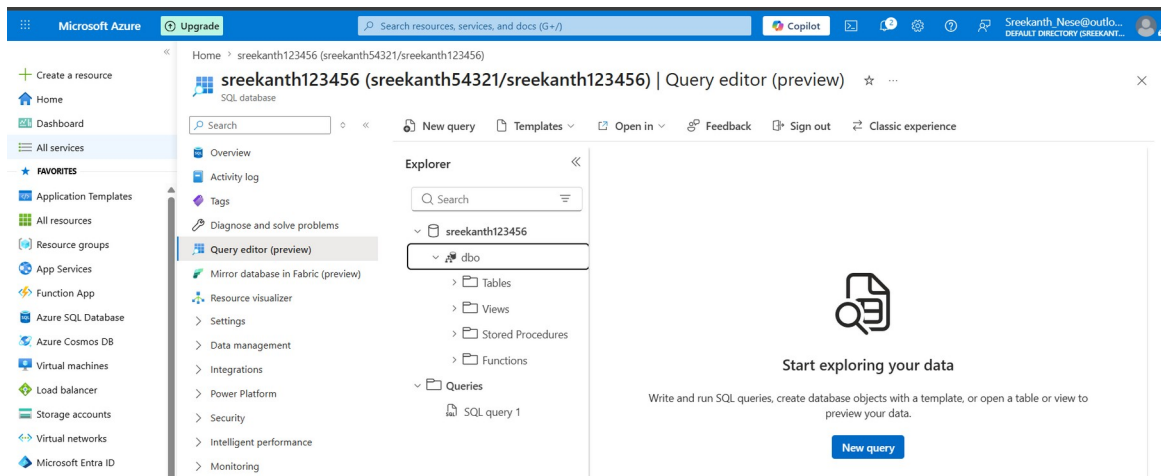
Step 3: Created the Azure Databricks Workspace

An Azure Databricks Service (sreekanth123456) was provisioned on the Premium pricing tier in West US, with public IP access disabled for the workspace. This is where the notebook and its Spark cluster run.



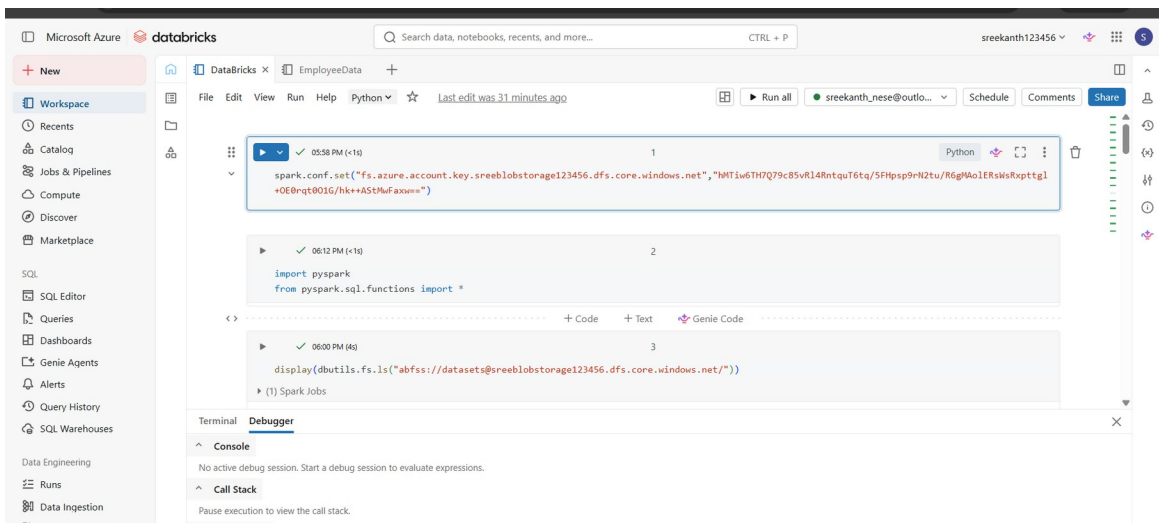
Step 4: Created the Azure SQL Database

An Azure SQL Database (sreekanth123456, on server sreekanth54321) was created in Australia East as the landing zone for every table this notebook produces — accessed later through the same Query Editor used in the other ADF projects.

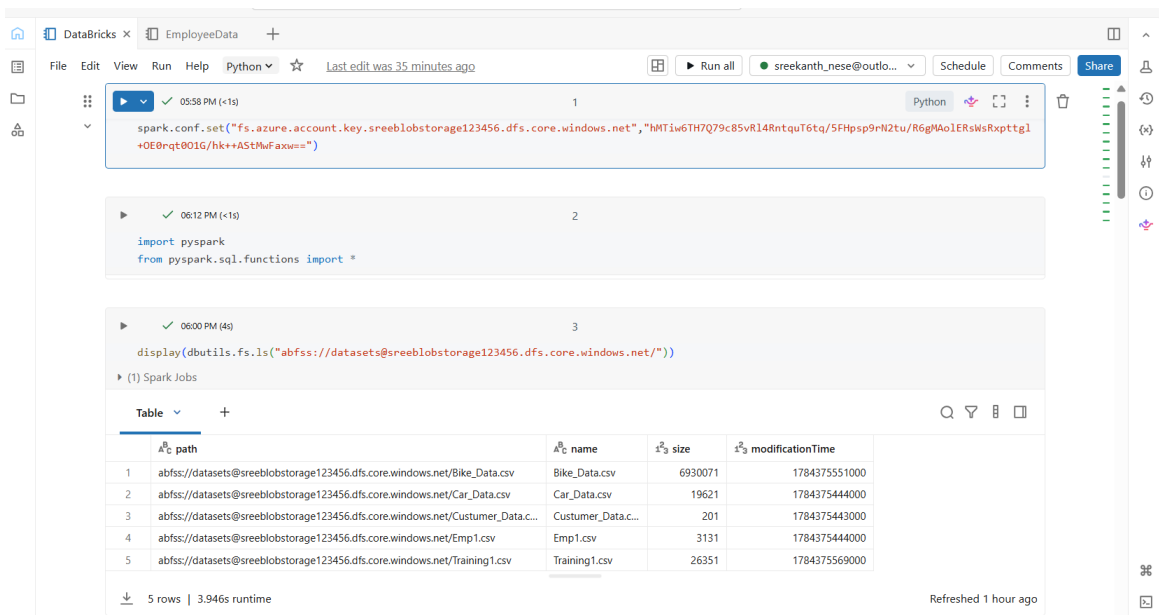


Step 5: Connected the Notebook to Blob Storage

In a notebook named EmployeeData, the first cell configures Spark to authenticate directly against the storage account using its account key (spark.conf.set("fs.azure.account.key.sreeblobstorage123456.dfs.core.windows.net", ...)), then imports pyspark and pyspark.sql.functions. A quick dbutils.fs.ls(...) call lists everything inside the datasets container to confirm the connection works.



The listing confirms all five files are visible to the notebook, along with their size and last-modified time — Bike_Data.csv, Car_Data.csv, Customer_Data.csv, Emp1.csv, and Training1.csv.



Step 6: Loaded and Explored the Bike Sales Data

Bike_Data.csv was read into a Spark DataFrame (`data = spark.read.csv(..., header=True, inferSchema=True)`). A quick preview shows columns like Region, Country, Customer, BusinessSegment, Category, Model, and Color. Checking `data.count()`, `len(data.columns)` confirms the raw file loaded as 60,919 rows across 11 columns.

The screenshot shows a Databricks notebook with the following code and output:

```
data=spark.read.csv("abfss://datasets@sreeblobstorage123456.dfs.core.windows.net/Bike_Data.csv",header=True,inferSchema=True)
```

(2) Spark Jobs

```
data: pyspark.sql.connect.dataframe.DataFrame = [Region: string, Country: string ... 9 more fields]
```

```
display(data.limit(5))
```

(1) Spark Jobs

	Region	Country	Customer	BusinessSegment	Category	Model	Color
1	North America	United States	Professional Containers and Packaging C...	Clothing	Tights	Women's Tights	Black
2	North America	United States	Professional Containers and Packaging C...	Clothing	Gloves	Full-Finger Gloves	Black
3	North America	United States	Professional Containers and Packaging C...	Bikes	Mountain Bikes	Mountain-200	Black
4	North America	United States	Professional Containers and Packaging C...	Clothing	Jerseys	Long-Sleeve Logo Jers...	Multi
5							

5 rows | 0.837s runtime Refreshed 1 hour ago

Step 7: Engineered New Business Columns

Three calculated columns were added directly on the Spark DataFrame: Sales ($\text{ListPrice} \times \text{OrderQty}$), Cost ($\text{UnitPrice} \times \text{OrderQty}$), and Profit ($\text{Sales} - \text{Cost}$). Re-checking the shape afterward shows the same 60,919 rows now carry 14 columns — the original 11 plus these 3 derived metrics.

The screenshot shows a Databricks notebook with the following code and output:

```
data.count(), len(data.columns)
```

(2) Spark Jobs

```
(60919, 11)
```

```
data=data.withColumn("Sales", col("ListPrice")*col("OrderQty"))
data=data.withColumn("Cost", col("UnitPrice")*col("OrderQty"))
data=data.withColumn("Profit", col("Sales")-col("Cost"))
```

data: pyspark.sql.connect.dataframe.DataFrame = [Region: string, Country: string ... 12 more fields]

```
data.count(), len(data.columns)
```

(2) Spark Jobs

```
(60919, 14)
```

```
display(data.limit(5))
```

(1) Spark Jobs

A `type(data)` check confirms this is still a proper Spark DataFrame (`pyspark.sql.connect.dataframe.DataFrame`) rather than a pandas object, right before the notebook moves on to writing the results out to Azure SQL Database (marked by the comment `#transfer to azure database`).

Table

	Region	Country	Customer	BusinessSegment	Category	Model	Color
1	North America	United States	Professional Containers and Packaging C...	Clothing	Tights	Women's Tights	Black
2	North America	United States	Professional Containers and Packaging C...	Clothing	Gloves	Full-Finger Gloves	Black
3	North America	United States	Professional Containers and Packaging C...	Bikes	Mountain Bikes	Mountain-200	Black
4	North America	United States	Professional Containers and Packaging C...	Clothing	Jerseys	Long-Sleeve Logo Jers...	Multi
5							

5 rows | 1.013s runtime

Refreshed 1 hour ago

06:20 PM (1s)

Run all

sreekanth_nese@outlo...

Schedule

Comments

Share

File Edit View Run Help Python Last edit was 36 minutes ago

(1) Spark Jobs

type(data)

pyspark.sql.connect.dataframe.DataFrame

#transfer to azure database

Step 8: Connected to Azure SQL Database via JDBC

A JDBC connection was defined using the SQL Server hostname (sreekanth54321.database.windows.net), port 1433, the target database name (sreekanth123456), and a DBProperties dictionary holding the SQL login. These pieces are combined into a JDBC URL

(jdbc:sqlserver://sreekanth54321.database.windows.net:1433;databaseName=sreekanth123456), which every subsequent .write.jdbc(...) call in the notebook reuses.

06:32 PM (<1s)

12

```

HostName="sreekanth54321.database.windows.net"
PortNum="1433"
DatabaseName="sreekanth123456"
DBProperties={
  "user": "sree",
  "password": "madhu@163"
}

```

06:35 PM (<1s)

13

```

print(HostName)
print(DBProperties)

```

sreekanth54321.database.windows.net

```

{'user': 'sree', 'password': 'madhu@163'}

```

06:43 PM (<1s)

14

```

urlOfTarget="jdbc:sqlserver://{0}:{1};databaseName={2}".format(HostName,PortNum,DatabaseName)

```

06:43 PM (<1s)

15

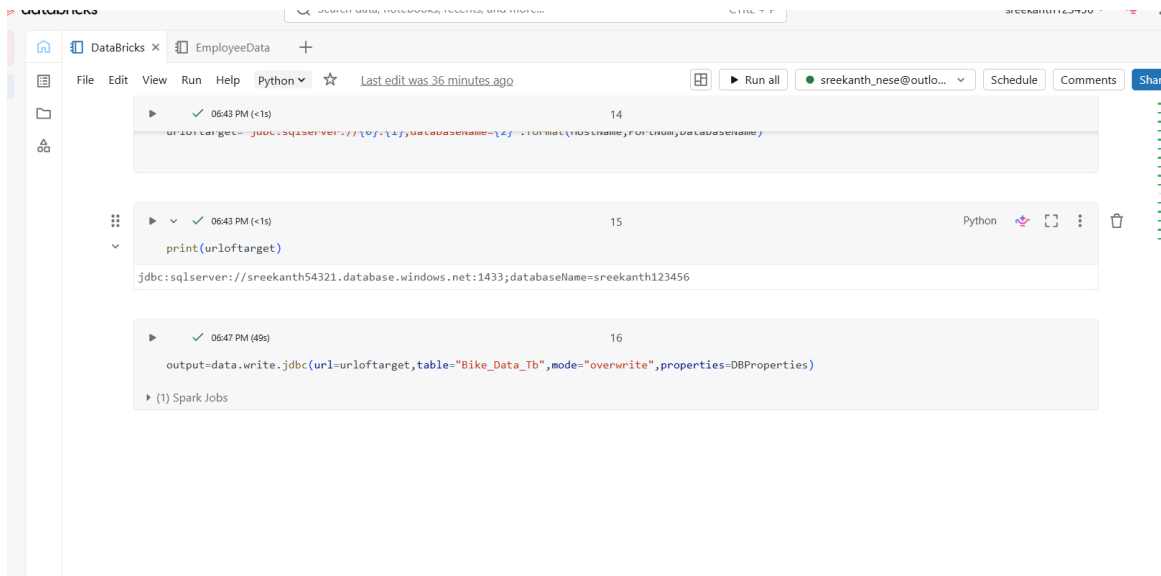
```

print(urlOfTarget)

```

jdbc:sqlserver://sreekanth54321.database.windows.net:1433;databaseName=sreekanth123456

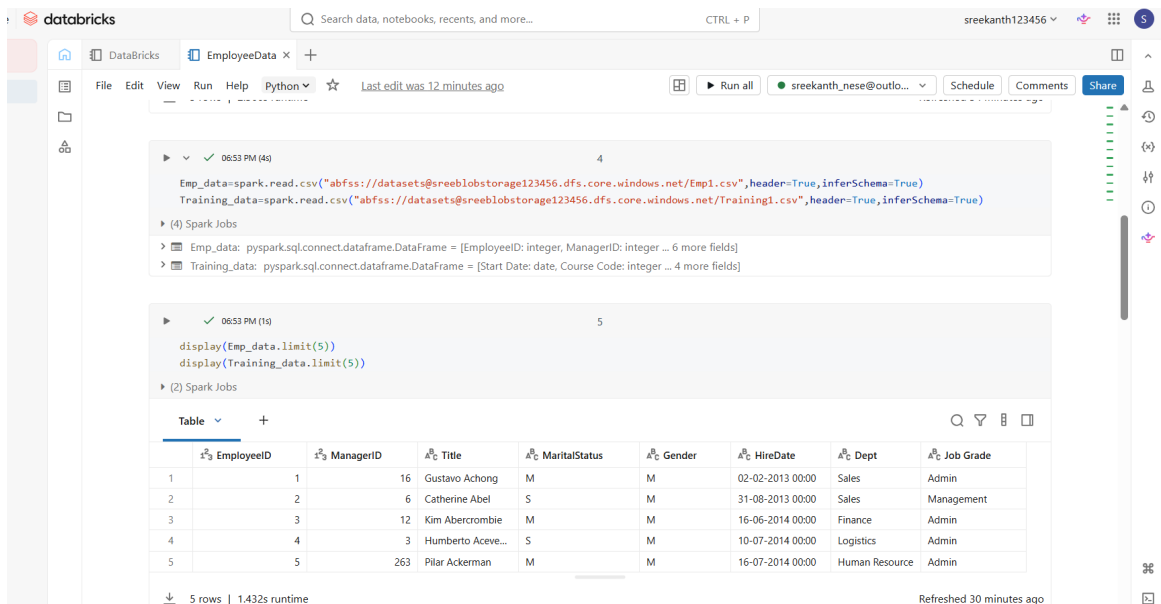
The enriched Bike Data DataFrame (with Sales, Cost, and Profit) is then written to Azure SQL Database over that JDBC connection, landing as the Bike_Data_Tb table with mode="overwrite".



Note: The notebook defines the SQL username and password directly as plain text in a code cell (`DBProperties = {"user": ..., "password": ...}`). For anything beyond a personal/learning project, storing that password in a Databricks secret scope (or Azure Key Vault, as used in the ADF projects) instead of inline in the notebook would keep the credential out of the notebook's source and revision history.

Step 9: Loaded the Employee and Training Data

Separately, `Emp1.csv` and `Training1.csv` were each read into their own Spark DataFrames — `Emp_data` (EmployeeID, ManagerID, Title, MaritalStatus, Gender, HireDate, Dept, Job Grade) and `Training_data` (Start Date, Course Code, Course Name, Employee ID, Cost, Supplier) — both with `header=True`, `inferSchema=True`.



Step 10: Joined and Aggregated the Employee Training Data

The two DataFrames are joined on employee ID (`Emp_data.EmployeeID == Training_data.Employee_ID`, an inner join) to produce `Empl_Training_Data`, a row-level combination of each employee with every training record they took. A second calculation, `Empl_Cost_Summary`, groups the same joined data by Title and sums the Cost column, then sorts descending — giving total training spend per job title, highest first.

Table

	Start Date	Course Code	Course Name	Employee_ID	Cost	Supplier
1	2015-09-18		1 Communication Worksh...		1 1300	Communication Expe...
2	2015-09-18		1 Communication Worksh...		4 1300	Communication Expe...
3	2015-09-18		1 Communication Worksh...		25 1300	Communication Expe...
4	2015-09-18		1 Communication Worksh...		33 1300	Communication Expe...
5	2015-09-18		1 Communication Worksh...		44 1300	Communication Expe...

5 rows | 1.432s runtime

Refreshed 30 minutes ago

```

6
Empl_Training_Data=Emp_data.join(Training_data,Emp_data.EmployeeID==Training_data.Employee_ID,"inner").select("*")

Empl_Training_Data: pyspark.sql.connect.dataframe.DataFrame = [EmployeeID: integer, ManagerID: integer ... 12 more fields]

7
Empl_Cost_Summary= Emp_data.join(Training_data,Emp_data.EmployeeID==Training_data.Employee_ID,"inner")\
    .groupBy("Title").agg(sum("Cost").alias("Total_Cost")).orderBy("Total_Cost",ascending=False)

Empl_Cost_Summary: pyspark.sql.connect.dataframe.DataFrame = [Title: string, Total_Cost: long]

```

Previewing Empl_Training_Data confirms the join worked as expected, combining each employee's HR details (Title, MaritalStatus, Gender, HireDate, Dept, Job Grade) with their training records. The same JDBC connection variables (HostName, PortNum, DatabaseName, DBProperties) are re-declared here ahead of the write step.

Table

	EmployeeID	ManagerID	Title	MaritalStatus	Gender	HireDate	Dept	Job Grade	
1		16	Gustavo Achong	M	M	02-02-2013 00:00	Sales	Admin	20'
2		3	Humberto Aceve...	S	M	10-07-2014 00:00	Logistics	Admin	20'
3		25	Phyllis Allen	S	M	09-07-2015 00:00	Production	Operations	20'
4		33	Maxwell Amland	S	M	13-07-2015 00:00	Sales	Operations	20'
5									

5 rows | 2.436s runtime

Refreshed 25 minutes ago

```

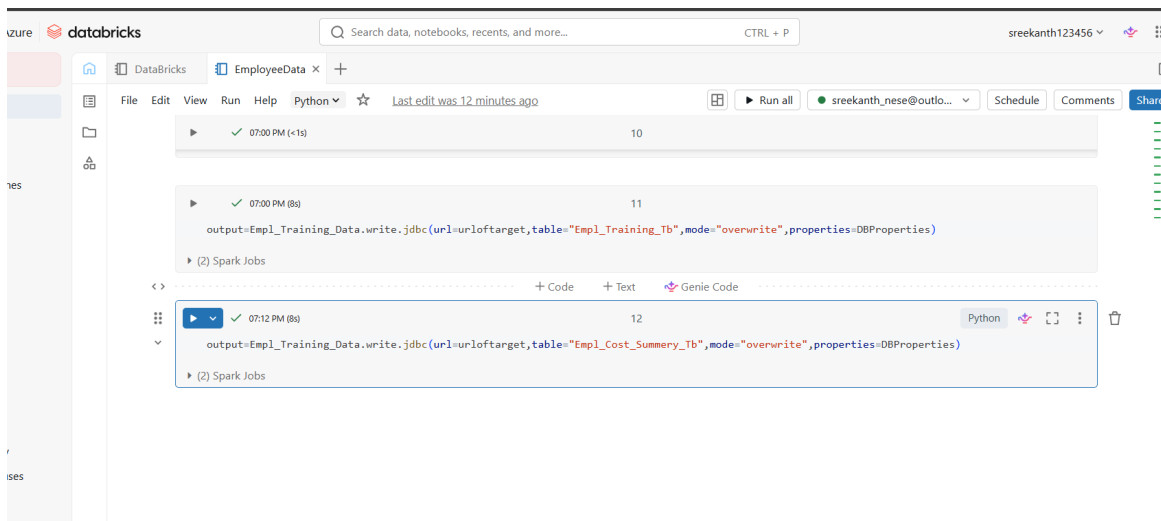
8
display(Empl_Training_Data.limit(5))

9
HostName="sreekanth54321.database.windows.net"
PortNum="1433"
DatabaseName="sreekanth123456"
DBProperties={
  "user": "sree",
  "password": "madhu@163"
}

```

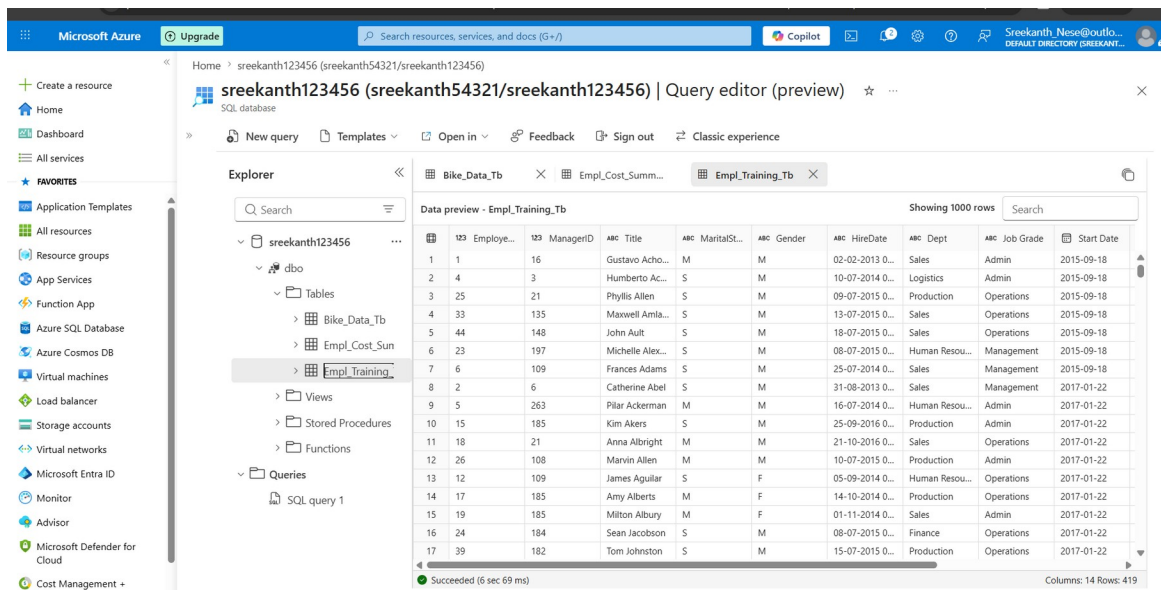
Step 11: Wrote the Employee Analysis Tables to Azure SQL

Two more JDBC writes land the employee-side results in Azure SQL Database: the joined employee/training data goes to Emp1_Training_Tb, and the Title-level cost aggregation goes to Emp1_Cost_Summary_Tb, both written with mode="overwrite".



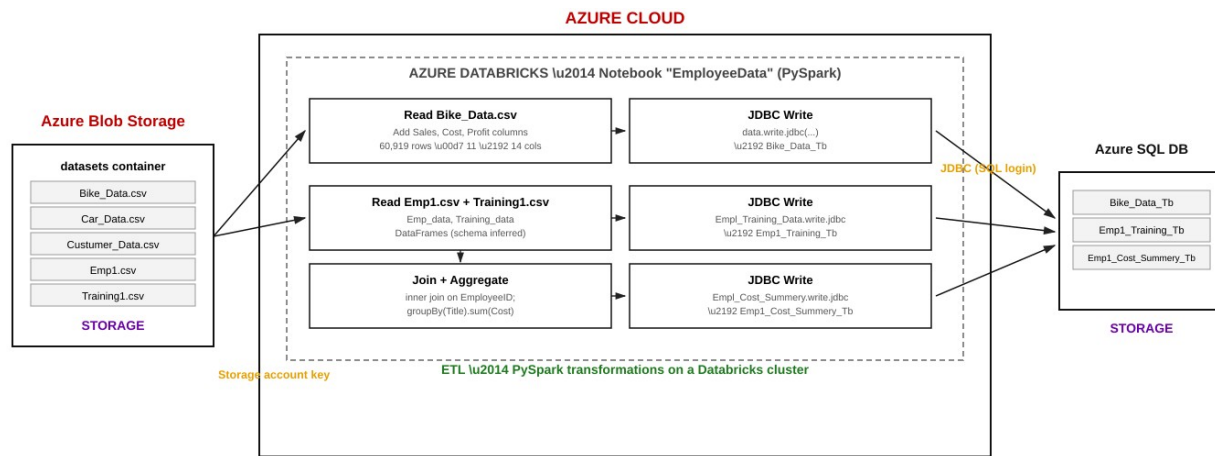
Step 12: Validated All Three Tables in Azure SQL Database

The Azure SQL Query Editor confirms all three output tables exist side by side — Bike_Data_Tb, Empl_Cost_Summary_Tb, and Empl_Training_Tb. Previewing Empl_Training_Tb shows the joined employee + training columns landed correctly, with 14 columns and 419 rows returned — confirming the join, aggregation, and JDBC write-back all worked end to end.



Architecture Diagram

The diagram below summarizes the pipeline: Blob Storage holds the five raw CSVs, the Databricks notebook authenticates against storage with the account key and runs two parallel PySpark workflows (Bike Data transformation, and Employee/Training join + aggregation), and each result is written over JDBC into its own table in Azure SQL Database.



Key Takeaway

Where the earlier ADF projects used Lookup/ForEach/Copy activities and Mapping Data Flows for transformation, this project shows the same kind of work — reading raw files, deriving new columns, joining and aggregating data, and landing the result in Azure SQL Database — done instead with PySpark code in a Databricks notebook. The JDBC write pattern (`data.write.jdbc(url=..., table=..., mode="overwrite", properties=...)`) is the Databricks equivalent of an ADF Copy activity's sink: however the transformation is done, it's the same JDBC connection that lands every output table in Azure SQL Database.